

Práctica 1

Parte 1

En esta primera parte de la práctica vamos a configurar el entorno de desarrollo que utilizaremos a lo largo del curso, que se compone de:

Visual Studio Code (VSCode): un entorno de desarrollo altamente adaptable y eficiente, reconocido por su interfaz intuitiva y su capacidad para integrar extensiones. Con funciones como resaltado de sintaxis y completado automático, VSCode proporciona un espacio de trabajo robusto y fácil de usar.

PlatformIO: una herramienta de desarrollo diseñada para simplificar la programación de sistemas empujados. Funciona como una extensión para diferentes IDEs, entre ellos VSCode, abstrayendo la complejidad de la configuración del toolchain y las tareas asociadas al desarrollo (compilación, carga, etc.) para diferentes placas ([boards](#)), arquitecturas ([platforms](#)), y entornos ([frameworks](#)).

Cada entorno es aplicable a diferentes arquitecturas; cada arquitectura está presente en diferentes placas.

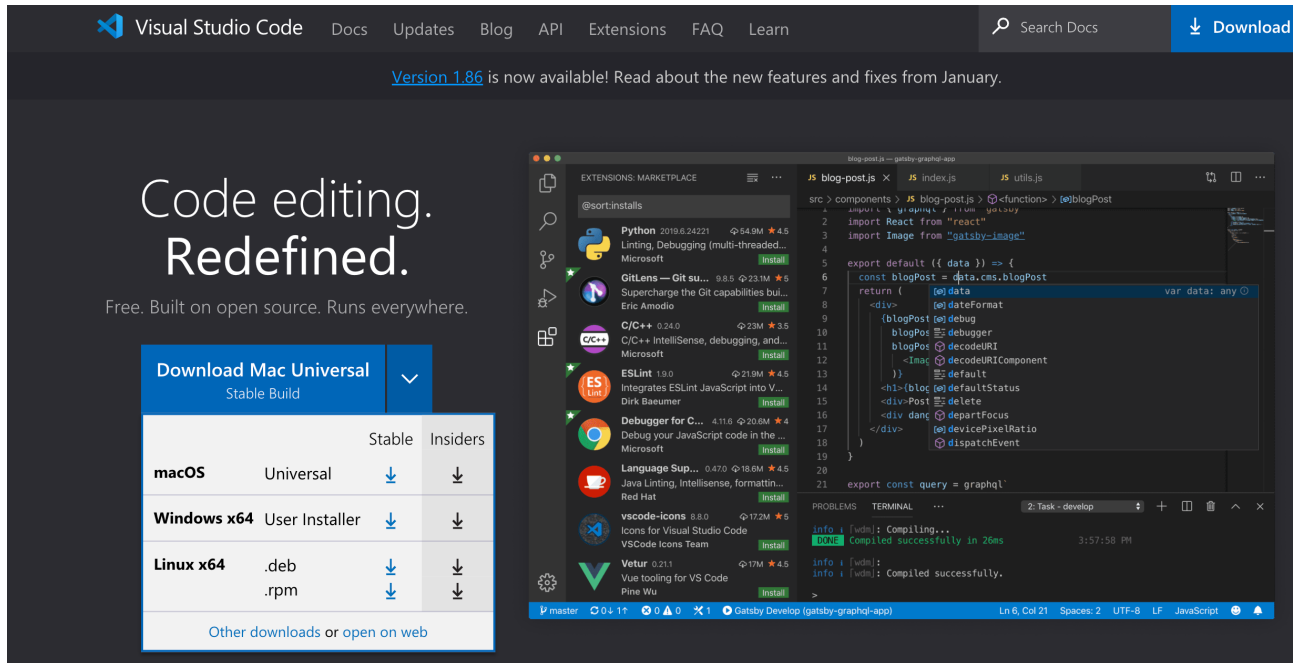
ESP-IDF (Espressif IoT Development Framework): es un entorno de desarrollo (un framework compatible en PlatformIO) dedicado para las arquitecturas de las placas del fabricante (esencialmente ESP32 y ESP8266 en sus diferentes versiones). Proporciona kernel, drivers, librerías y el toolchain (compilador, linker, bootloader, etc.) para programar sus placas, así como herramientas front-end para su fácil utilización.

FreeRTOS: un sistema operativo en tiempo real apropiativo multitarea de código abierto, conocido por su diseño modular. Ofrece programación multitarea, semáforos y servicios esenciales para el desarrollo de sistemas eficientes en tiempo real.

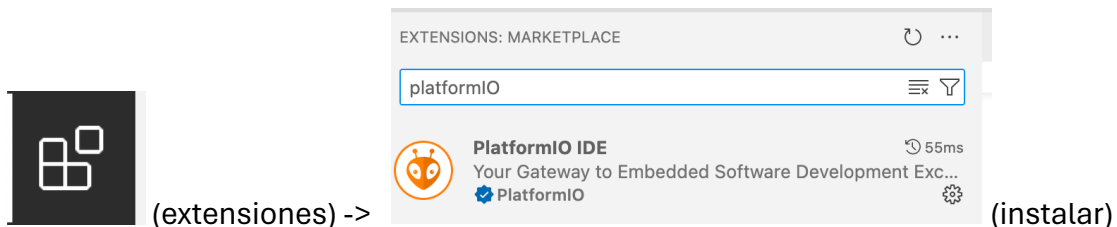
ESP-IDF utiliza una versión adaptada de FreeRTOS como Kernel (Vanilla FreeRTOS), proporcionando además librerías propias de Espressif que, a modo de wrapper de FreeRTOS, proporcionan abstracción para sacar el máximo partido a su hardware. Para saber más: [FreeRTOS \(IDF\)](#).

Para esta primera parte de la práctica tienes que escribir una memoria respondiendo a una serie de preguntas o aportando evidencia de algunas actividades. Estos apartados están indicados en rojo a lo largo del documento. Sigue los siguientes pasos:

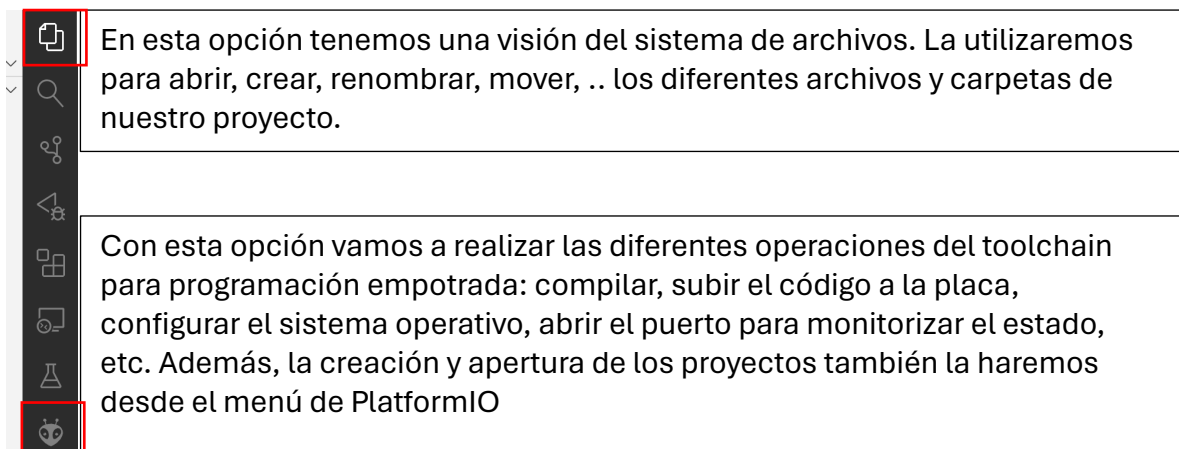
1. Entra en <https://code.visualstudio.com>, descarga el IDE para tu sistema operativo e instálalo en tu sistema.



2. Abre VSCode e instala la extensión PlatformIO.

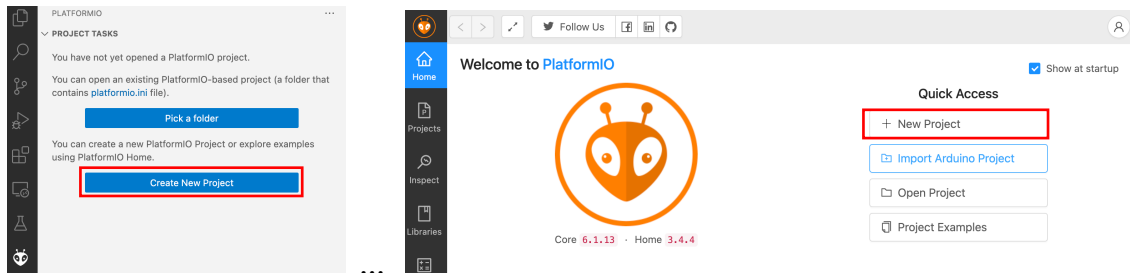


3. Una vez instalado, el entorno VSCode muestra una barra lateral como la siguiente:



Además de estas dos opciones, desde VSCode puedes utilizar funcionalidades del control de versiones (GIT), y tiene muchas más opciones interesantes, además del market de extensiones, te recomiendo que lo explores y experimentes libremente.

4. Es hora de crear nuestro primer proyecto de prueba. Entra en las opciones de PlatformIO y selecciona crear nuevo proyecto. En la parte derecha aparecerá PlatformIO Home (si no estaba desde el principio), selecciona nuevo proyecto en ese menú.



5. Rellena el formulario como aparece en la imagen.

Project Wizard
×

This wizard allows you to **create new** PlatformIO project or **update existing**. In the last case, you need to uncheck "Use default location" and specify path to existing project.

Name:

Board:

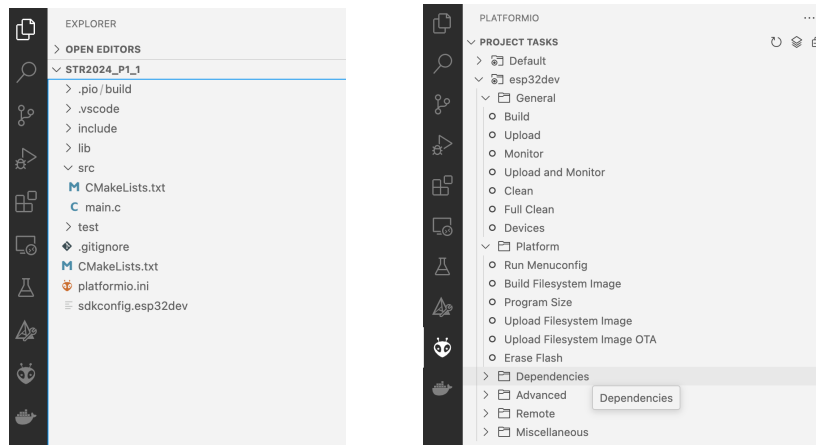
Framework:

Location: ☒ Use default location ?

Cancel
Finish

La ubicación por defecto suele ser <home>/Documents/PlatformIO/Projects. Puedes usar esta ubicación, o puedes crear una carpeta en tu espacio de usuario para la práctica. En cualquier caso, es importante que conozcas bien dónde se encuentra tu código.

6. Cuando finalice, selecciona las dos vistas: sistema de archivos (izquierda) y PlatformIO (derecha) y asegúrate de que tienes algo similar a lo siguiente.



Un proyecto ESP-IDF en PlatformIO para VSCode sigue una estructura organizativa como la siguiente:

- **.pio y .vscode:** Carpetas ocultas que contienen configuraciones internas de PlatformIO y VSCode, respectivamente. Puedes ignorar su contenido en la mayoría de los casos.
- **build:** Carpeta que almacena los archivos generados durante el proceso de compilación (esta carpeta aparecerá en el momento de construir el binario). Aquí encontrarás el firmware compilado, así como archivos temporales y de construcción.
- **include y src:** Carpetas que contienen los archivos de encabezado (.h) y de código fuente (.c o .cpp), respectivamente, para tu proyecto principal. Aquí es donde escribirás la lógica principal de tu aplicación.
- **lib:** Carpeta opcional que puede contener bibliotecas de terceros o bibliotecas personalizadas que deseas incluir en tu proyecto.
- **test:** Carpeta opcional que puede contener pruebas unitarias para tu código.
- **CMakeLists.txt:** Archivo de configuración de CMake utilizado para describir cómo se debe construir tu proyecto.
- **platformio.ini:** Archivo de configuración de PlatformIO. Aquí especificas las configuraciones del proyecto, como la placa objetivo, la velocidad de baudios, y otras opciones relacionadas con la compilación y carga del firmware.

*Tómate tu tiempo para explorar estas carpetas y los ficheros que contienen. Observa que, de momento, main.c solo contiene una función vacía definida como **void app_main()**. Este es el punto de entrada de los proyectos con ESP-IDF, y a partir de ella construiremos nuestro código.*

7. Vamos a introducir algo de código en el proyecto. Abre el fichero main.c (desde la vista de sistema de archivos) e introduce el siguiente código, cambiando <nombre> y <apellidos> por tu nombre y tus apellidos, respectivamente:

```
#include <stdio.h>
#include "esp_log.h"
#include "esp_chip_info.h"
#include "esp_clk_tree.h"

const char* TAG = "P1_1";
```

```
// Función para obtener el nombre del modelo como cadena
const char* get_chip_model_name(esp_chip_model_t model) {
    switch (model) {
        case CHIP_ESP32:         return "ESP32";
        case CHIP_ESP32S2:       return "ESP32-S2";
        case CHIP_ESP32S3:       return "ESP32-S3";
        case CHIP_ESP32C3:       return "ESP32-C3";
        case CHIP_ESP32C2:       return "ESP32-C2";
        case CHIP_ESP32C6:       return "ESP32-C6";
        case CHIP_ESP32H2:       return "ESP32-H2";
        case CHIP_POSIX_LINUX:   return "POSIX/Linux Simulator";
        default:                 return "Unknown";
    }
}

void app_main(void)
{
    // Estructura para almacenar información del chip
    esp_chip_info_t chip_info;
    // Entero para almacenar la frecuencia de la CPU
    uint32_t freq;

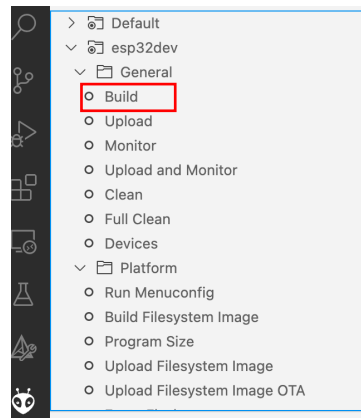
    ESP_LOGI(TAG, "Hello ESP32!");
    ESP_LOGI(TAG, "<Nombre1>, <Apellidos1>");
    ESP_LOGI(TAG, "<Nombre2>, <Apellidos2>");
    // ...
    // ESP_LOGI(TAG, "<NombreN>, <ApellidosN>");

    // Obtener información del chip
    esp_chip_info(&chip_info);
    if (esp_clk_tree_src_get_freq_hz(SOC_MOD_CLK_CPU,
    ESP_CLK_TREE_SRC_FREQ_PRECISION_EXACT, &freq) != ESP_OK)
    {
        ESP_LOGI(TAG, "Error obteniendo la frecuencia de la CPU");
        freq = 0;
    }

    // Formatear la cadena con el modelo de chip y el número de cores
    ESP_LOGI(TAG, "Modelo: %s, Cores: %d, Frecuencia: %ld",
    get_chip_model_name(chip_info.model), chip_info.cores, freq);
}
```

El entorno integrado de VSCode te permite navegar por el código hacia sus definiciones. Pulsa la letra CONTROL y clic en "esp_chip_info.h" para navegar al fichero de cabecera correspondiente. También puedes navegar con grano más fino haciendo lo propio con el nombre de las estructuras, prueba a seleccionar "esp_chip_info_t".

8. Para compilar el proyecto, desde el menú de PlatformIO, selecciona "build".



El proceso de compilación con PlatformIO abrirá una terminal de salida en la parte inferior derecha del entorno. Si todo va correctamente, finalmente obtendrás un binario (.elf) que la arquitectura del chip puede ejecutar, solo falta escribirlo en la flash del chip.

Al inicio del log de compilación, PlatformIO te da información específica acerca del hardware y del entorno, como las versiones que está utilizando de las diferentes dependencias.

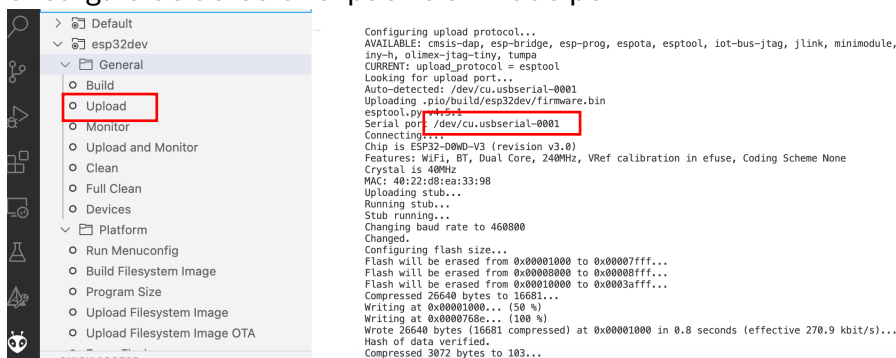
Al final del log te da información del firmware que se ha generado así como del porcentaje de uso de la memoria que supone.

Apartado I: Primera compilación.

Responde a las siguientes preguntas y respáldalas con capturas de pantalla:

- 1.- ¿Cuánta capacidad tiene la flash del hardware que estás utilizando?
- 2.- ¿Qué versión de espidf tiene el entorno que ha creado PlatformIO?
- 3.- ¿Cuánta memoria Flash ocupa el firmware generado? ¿Qué porcentaje supone del disponible?
- 4.- ¿Cuánta memoria RAM supone la ejecución de este código? ¿Qué porcentaje supone del disponible?
- 5.- ¿En qué ruta está el fichero .elf generado?

9. Para grabar este firmware en la placa, basta con conectarla con USB a nuestro ordenador y seleccionar la opción “upload” de PlatformIO. El propio entorno se encargará de detectar el puerto utilizado por ti.



Si esta operación te da error, es posible que no tengas instalados los drivers del convertidor UART-USB que utilizan las placas ESP32. Descárgalo e instálalo para tu sistema operativo desde aquí:

<https://www.silabs.com/developers/usb-to-uart-bridge-vcp-drivers?tab=downloads>



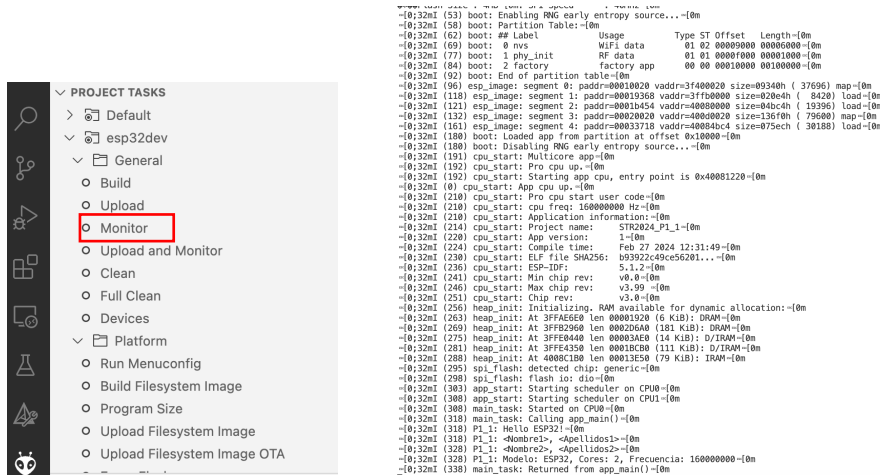
10. Vamos a configurar el entorno para que utilice siempre el mismo puerto, esto evitará problemas en el caso de que conectes otros dispositivos a la vez. También vamos a definir la velocidad de transferencia de bytes a través de la UART (baudios). Abre el fichero platformio.ini y añade la siguiente línea:

```
upload_port = <puerto detectado>
monitor_port = <puerto detectado>
monitor_speed = 115200
```

En el ejemplo anterior sería:

```
upload_port = /dev/cu.usbserial-0001
monitor_port = /dev/cu.usbserial-0001
monitor_speed = 115200
```

11. Para monitorizar la actividad por el Puerto serie, utiliza la opción monitor desde el menu de PlatformIO.



Observa que al iniciar el monitor, el sistema se reinicia. También puedes reiniciarlo manualmente las veces que quieras pulsando el botón marcado como EN en la placa. ¡Prueba!



Apartado II: Primera ejecución.

Responde a las siguientes preguntas y respáldalas con capturas de pantalla:

- 1.- Haz una captura de pantalla de la salida por monitor que te da la aplicación.
- 2.- ¿Qué tipo de información te ofrece el log de inicio del sistema?